BUBBLE SHOOTER GAME DEVELOPMENT USING PYTHON

AIM

To design and develop an interactive 2D Bubble Shooter game using Python and Tkinter. The player aims and shoots colored bubbles to match three or more bubbles of the same color, earns points, and clears bubbles from a galaxy-themed game board. The game includes mouse controls, score and shot tracking, bubble placement, win/game-over detection, and restart functionality.

PROCEDURE

1. Import tkinter, random, and math libraries.
2. Create the main window and configure its size, title, and space-themed background.
3. Create a Tkinter Canvas for the board, stars, planets, bubbles, launcher, and interface.
4. Generate colored bubbles and store their grid positions and colors.
5. Display the current bubble, next bubble, score, and number of shots.
6. Use mouse movement to calculate the aiming angle and display an aiming guide.
7. Animate the fired bubble toward the top of the board, reflecting it from side walls.
8. Detect contact with the ceiling or another bubble and attach the bubble to the nearest empty grid cell.
9. Find connected bubbles of the same color and remove the group when three or more match.
10. Award points for matched bubbles and remove bubbles disconnected from the top row.
11. Check if the board is cleared or bubbles reach the launcher area; display the result.
12. Allow the player to restart by pressing R or clicking the restart area

PROGRAM CODE

```
import tkinter as tk
import random
import math

# ------------------------------------------------------------
# SPACE BUBBLE SHOOTER
# Controls: Move the mouse to aim, click to shoot, R to restart.
# ------------------------------------------------------------

WIDTH = 720
HEIGHT = 760
BG = "#080d2b"
PANEL = "#101943"
BUBBLE_R = 18
COLORS = ["#ff4f91", "#31d7ff", "#8b5cf6", "#50e36b", "#ffbf3f"]
SHOOTER_X = WIDTH // 2
SHOOTER_Y = HEIGHT - 70
TOP = 100
ROWS = 8
COLS = 17
STEP_X = BUBBLE_R * 2
STEP_Y = int(BUBBLE_R * 1.72)
LEFT = (WIDTH - (COLS * STEP_X - BUBBLE_R)) // 2
MAX_SHOTS = 0


class SpaceBubbleShooter:
    def __init__(self, root):
        self.root = root
        self.root.title("Space Bubble Shooter")
        self.root.resizable(False, False)
        self.root.configure(bg=BG)

        self.canvas = tk.Canvas(
            root, width=WIDTH, height=HEIGHT, bg=BG,
            highlightthickness=0
        )
        self.canvas.pack()
        self.canvas.bind("<Motion>", self.aim)
        self.canvas.bind("<Button-1>", self.shoot)
        self.root.bind("<KeyPress-r>", lambda event: self.restart())
        self.root.bind("<KeyPress-R>", lambda event: self.restart())

        self.restart()
```

```
def restart(self):
        self.canvas.delete("all")
        self.score = 0
        self.shots = 0
        self.game_over = False
        self.won = False
        self.angle = -math.pi / 2
        self.bubbles = {}
        self.make_stars()
        self.make_board()
        self.make_header()

        for row in range(ROWS):
            for col in range(COLS):
                # A gently varied lower edge makes the opening board feel less rigid.
                if row >= 6 and random.random() < 0.28:
                    continue
                color = random.choice(COLORS)
                self.bubbles[(row, col)] = color

        self.current_color = random.choice(COLORS)
        self.next_color = random.choice(COLORS)
        self.draw_all()
        self.draw_shooter()
        self.draw_hud()

    def make_stars(self):
        for _ in range(100):
            x = random.randint(0, WIDTH)
            y = random.randint(0, HEIGHT)
            size = random.choice([1, 1, 2, 2, 3])
            color = random.choice(["#26366d", "#4056a6", "#7385d6", "#ffffff"])
            self.canvas.create_oval(x, y, x + size, y + size,
                                    fill=color, outline="")
        # Decorative planets
        self.canvas.create_oval(22, 120, 76, 174, fill="#182c70",
                                outline="#4269d9", width=2)
        self.canvas.create_arc(15, 130, 83, 164, start=190, extent=160,
                               style="arc", outline="#8d74ff", width=4)
        self.canvas.create_oval(640, 160, 681, 201, fill="#321c68",
                                outline="#a16cff", width=2)
        for x, y in [(85, 240), (615, 300), (80, 500), (650, 540)]:
            self.canvas.create_text(x, y, text="✦", fill="#526bd1",
                                    font=("Arial", 16, "bold"))
```

```
def make_board(self):
    self.canvas.create_rectangle(24, 92, WIDTH - 24, HEIGHT - 22,
                                 outline="#294caa", width=2)
    self.canvas.create_line(25, TOP + 8, WIDTH - 25, TOP + 8,
                            fill="#3459c7", width=2)

def make_header(self):
    self.canvas.create_text(WIDTH // 2, 34, text="✦  SPACE BUBBLE SHOOTER  ✦",
                            fill="#f4f6ff", font=("Arial", 21, "bold"))
    self.canvas.create_text(WIDTH // 2, 64,
                            text="MATCH 3 OR MORE • CLEAR THE GALAXY",
                            fill="#7ddcff", font=("Arial", 10, "bold"))

def draw_hud(self):
    self.canvas.create_rectangle(40, 103, 155, 137, fill=PANEL,
                                 outline="#3e65d8", width=1, tags="hud")
    self.canvas.create_text(97, 120, text=f"SCORE  {self.score}",
                            fill="#ffffff", font=("Arial", 11, "bold"),
                            tags="hud")
    self.canvas.create_rectangle(WIDTH - 148, 103, WIDTH - 40, 137,
                                 fill=PANEL, outline="#3e65d8", width=1,
                                 tags="hud")
    self.canvas.create_text(WIDTH - 94, 120, text=f"SHOTS  {self.shots}",
                            fill="#ffffff", font=("Arial", 11, "bold"),
                            tags="hud")
    self.canvas.create_text(WIDTH // 2, HEIGHT - 15,
                            text="MOUSE TO AIM  •  CLICK TO FIRE  •  R TO RESTART",
                            fill="#8193db", font=("Arial", 9, "bold"),
                            tags="hud")

def draw_bubble(self, x, y, color, radius=BUBBLE_R, tags=()):
    # A glossy orb with a small highlight.
    self.canvas.create_oval(x-radius, y-radius, x+radius, y+radius,
                            fill=color, outline="#d9e7ff", width=1,
                            tags=tags)
    self.canvas.create_oval(x-radius*0.52, y-radius*0.62,
                            x-radius*0.08, y-radius*0.18,
                            fill="#ffffff", outline="", tags=tags)

def center(self, cell):
    row, col = cell
    x = LEFT + col * STEP_X + (BUBBLE_R if row % 2 else 0)
    y = TOP + row * STEP_Y + BUBBLE_R
    return x, y
```

```
    def draw_all(self):
        self.canvas.delete("bubble")
        for cell, color in self.bubbles.items():
            x, y = self.center(cell)
            self.draw_bubble(x, y, color, tags=("bubble",))

    def draw_shooter(self):
        self.canvas.delete("shooter")
        # Aim guide, reflected at the side walls.
        sx, sy = SHOOTER_X, SHOOTER_Y
        dx, dy = math.cos(self.angle), math.sin(self.angle)
        ex, ey = sx + dx * 230, sy + dy * 230
        self.canvas.create_line(sx, sy, ex, ey, fill="#6d8cff",
                                width=2, dash=(3, 7), tags="shooter")
        self.canvas.create_oval(sx-35, sy-35, sx+35, sy+35,
                                fill="#182b70", outline="#4d7cff",
                                width=3, tags="shooter")
        self.draw_bubble(sx, sy, self.current_color, radius=22,
                         tags=("shooter",))
        self.canvas.create_text(sx, sy+43, text="FIRE",
                                fill="#a7c5ff", font=("Arial", 9, "bold"),
                                tags="shooter")
        # Next bubble preview
        self.canvas.create_rectangle(WIDTH-115, HEIGHT-116, WIDTH-36,
                                     HEIGHT-48, fill=PANEL,
                                     outline="#3e65d8", tags="shooter")
        self.canvas.create_text(WIDTH-75, HEIGHT-103, text="NEXT",
                                fill="#9cb8ff", font=("Arial", 8, "bold"),
                                tags="shooter")
        self.draw_bubble(WIDTH-75, HEIGHT-75, self.next_color, radius=15,
                         tags=("shooter",))

    def aim(self, event):
        if self.game_over:
            return
        dx = event.x - SHOOTER_X
        dy = event.y - SHOOTER_Y
        if dy > -15:
            dy = -15
        self.angle = math.atan2(dy, dx)
        # Keep aim pointed into the upper half of the board.
        self.angle = max(-math.pi + 0.12, min(-0.12, self.angle))
        self.draw_shooter()

    def neighbors(self, cell):
        row, col = cell
        if row % 2 == 0:
            offsets = [(-1, -1), (-1, 0), (0, -1), (0, 1),
                       (1, -1), (1, 0)]
        else:
            offsets = [(-1, 0), (-1, 1), (0, -1), (0, 1),
                       (1, 0), (1, 1)]
        result = []
        for dr, dc in offsets:
            nr, nc = row + dr, col + dc
            if 0 <= nr < 30 and 0 <= nc < COLS:
                result.append((nr, nc))
        return result
```

```
    def shoot(self, event=None):
        if self.game_over:
            return
        x, y = float(SHOOTER_X), float(SHOOTER_Y)
        vx = math.cos(self.angle) * 9
        vy = math.sin(self.angle) * 9

        # Animate the projectile in small steps, reflecting off side walls.
        def travel():
            nonlocal x, y, vx, vy
            if self.game_over:
                return
            x += vx
            y += vy
            if x < BUBBLE_R + 25:
                x = BUBBLE_R + 25
                vx = abs(vx)
            elif x > WIDTH - BUBBLE_R - 25:
                x = WIDTH - BUBBLE_R - 25
                vx = -abs(vx)

            hit = y <= TOP + BUBBLE_R
            if not hit:
                for cell in self.bubbles:
                    bx, by = self.center(cell)
                    if math.hypot(x - bx, y - by) <= BUBBLE_R * 2 - 2:
                        hit = True
                        break

            if hit:
                self.land_bubble(x, y)
                return

            self.canvas.delete("projectile")
            self.draw_bubble(x, y, self.current_color, radius=13,
                             tags=("projectile",))
            self.root.after(12, travel)

        self.canvas.delete("projectile")
        travel()

    def land_bubble(self, x, y):
        # Attach to the nearest empty grid cell.
        best_cell = None
        best_dist = float("inf")
        for row in range(30):
            for col in range(COLS):
                cell = (row, col)
```

```
            if cell in self.bubbles:
                continue
            cx, cy = self.center(cell)
            if cy < TOP + BUBBLE_R or cy > SHOOTER_Y - 45:
                continue
            dist = math.hypot(x-cx, y-cy)
            if dist < best_dist:
                best_dist = dist
                best_cell = cell

        if best_cell is None:
            self.end_game(False)
            return

        self.bubbles[best_cell] = self.current_color
        self.shots += 1
        matched = self.find_match(best_cell)
        if len(matched) >= 3:
            for cell in matched:
                self.bubbles.pop(cell, None)
            self.score += len(matched) * 10
            floating = self.find_floating()
            for cell in floating:
                self.bubbles.pop(cell, None)
            self.score += len(floating) * 15

        self.current_color = self.next_color
        remaining_colors = list(set(self.bubbles.values())) or COLORS
        self.next_color = random.choice(remaining_colors)
        self.draw_all()
        self.canvas.delete("projectile")
        self.canvas.delete("hud")
        self.draw_hud()
        self.draw_shooter()

        if not self.bubbles:
            self.end_game(True)
        elif any(self.center(cell)[1] >= SHOOTER_Y - 65
                 for cell in self.bubbles):
            self.end_game(False)

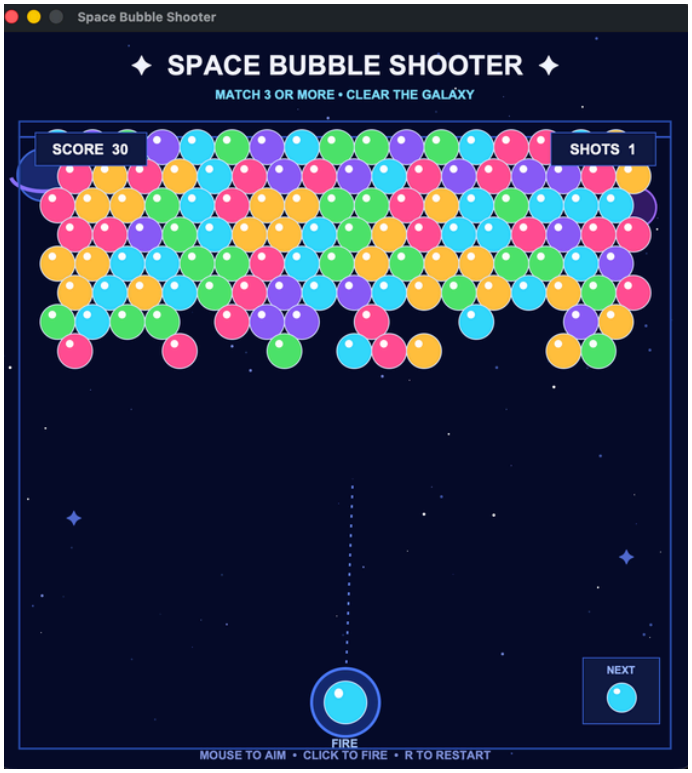
    def find_match(self, start):
        target = self.bubbles[start]
        stack = [start]
        found = set()
        while stack:
            cell = stack.pop()
            if cell in found or self.bubbles.get(cell) != target:
                continue
            found.add(cell)
            for neighbor in self.neighbors(cell):
                if neighbor not in found and self.bubbles.get(neighbor) == target:
                    stack.append(neighbor)
        return found
```

```
def find_floating(self):
        connected = set()
        stack = [cell for cell in self.bubbles if cell[0] == 0]
        while stack:
            cell = stack.pop()
            if cell in connected or cell not in self.bubbles:
                continue
            connected.add(cell)
            stack.extend(n for n in self.neighbors(cell)
                         if n in self.bubbles and n not in connected)
        return set(self.bubbles) - connected

    def end_game(self, won):
        self.game_over = True
        self.canvas.delete("overlay")
        self.canvas.create_rectangle(130, 300, WIDTH-130, 430,
                                     fill="#101943", outline="#6d8cff",
                                     width=3, tags="overlay")
        title = "GALAXY CLEARED!" if won else "MISSION OVER"
        color = "#65f4c2" if won else "#ff719d"
        self.canvas.create_text(WIDTH//2, 342, text=title, fill=color,
                                font=("Arial", 23, "bold"), tags="overlay")
        self.canvas.create_text(WIDTH//2, 378,
                                text=f"Final score: {self.score}",
                                fill="white", font=("Arial", 15, "bold"),
                                tags="overlay")
        self.canvas.create_text(WIDTH//2, 408,
                                text="Press R or click RESTART below",
                                fill="#a9baff", font=("Arial", 10),
                                tags="overlay")
        self.canvas.create_rectangle(WIDTH//2-65, 450, WIDTH//2+65, 489,
                                     fill="#315fe5", outline="#7ccaff",
                                     width=2, tags="overlay")
        self.canvas.create_text(WIDTH//2, 469, text="RESTART",
                                fill="white", font=("Arial", 12, "bold"),
                                tags="overlay")
        self.canvas.tag_bind("overlay", "<Button-1>",
                             lambda event: self.restart())


if __name__ == "__main__":
    root = tk.Tk()
    game = SpaceBubbleShooter(root)
    root.mainloop()
```

OUTPUT





RESULT:

Thus, an interactive 2D Space Bubble Shooter game was developed using Python Tkinter. The game demonstrates GUI design, mouse-based controls, animation using the after() method, bubble matching, connected-group detection, score calculation, and win/game-over handling